

P3124R0

2024-02 Library Evolution Poll Outcomes

Published Proposal, 2024-03-13

Authors:

[Inbal Levi - Library Evolution Chair](#) (MPGC Services LTD)

[Fabio Fracassi - Library Evolution Assistant Chair](#) (CODE University of Applied Sciences)

[Ben Craig - Library Evolution Assistant Chair](#) (Raven)

[Billy Baker - Library Evolution Incubator Chair](#) (NVIDIA)

[Nevin Liber - Library Evolution Incubator Assistant Chair and Admin Chair](#) (Argonne National Laboratory)

[Corentin Jabot - Library Mailing List Review Manager](#)

Source:

[GitHub](#)

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Audience:

WG21

Table of Contents

1 Introduction

2 Poll Outcomes

3 Selected Poll Comments

- 3.1 Poll 1: Send "[P3019R6] Vocabulary Types for Composite Class Design" to Library Working Group for C++26.
- 3.2 Poll 2: Send "[P2872R2] Remove `wstring_convert` From C++26" to Library Working Group for C++26.
- 3.3 Poll 3: Send "[P2875R3] Undeprecate `polymorphic_allocator::destroy` for C++26" to Library Working Group for C++26.
- 3.4 Poll 4: Send "[P3029R0] Better `mdspan`'s CTAD" to Library Working Group for C++26.

3.5 Poll 5: Approve "[P3109R0] A plan for `std::execution` for C++26".

References

Informative References

§ 1. Introduction

In 2024-02, the C++ Library Evolution group conducted a series of electronic decision polls [P3123R0]. This paper provides the results of those polls and summarizes the results.

In total, 19 people participated in the polls. Some participants opted to not vote on some polls (Poll 3 and Poll 4 had a low participation rate). Thank you to everyone who participated, and to the papers' authors for all their hard work!

§ 2. Poll Outcomes

- SF: Strongly Favor.
- WF: Weakly Favor.
- N: Neutral.
- WA: Weakly Against.
- SA: Strongly Against.

Poll	SF	WF	N	WA	SA	Outcome
Poll 1: Send "[P3019R6] Vocabulary Types for Composite Class Design" to Library Working Group for C++26.	9	3	3	0	2	Consensus in favor
Poll 2: Send "[P2872R2] Remove <code>wstring_convert</code> From C++26" to Library Working Group for C++26.	7	4	2	0	0	Strong consensus in favor
Poll 3: Send "[P2875R3] Undeprecate <code>polymorphic_allocator::destroy</code> for C++26" to Library Working Group for C++26.	3	6	0	0	0	Consensus in favor
Poll 4: Send "[P3029R0] Better <code>mdspan</code> 's CTAD" to Library Working Group for C++26.	5	4	1	0	1	Consensus in favor
Poll 5: Approve "[P3109R0] A plan for <code>std::execution</code> for C++26".	6	6	1	0	0	Strong consensus in

All the polls have consensus in favor, and all but P3109 (which is approved as a plan) will be forwarded to LWG.

§ 3. Selected Poll Comments

For some of the comments, small parts were removed to anonymize.

§ 3.1. Poll 1: Send "[P3019R6] Vocabulary Types for Composite Class Design" to Library Working Group for C++26.

Both `indirect` and `polymorphic` are excellent utilities. They make it easier to implement correct designs by avoiding duplicating code (e.g., using `unique_ptr`) and nontrivial special member functions. As a result, they promote the "Rule of Zero" and thus contribute to the overall trend of C++ becoming less verbose. These types will also make C++ more teachable, so that users won't need to understand `unique_ptr` and noncopyable types in general; users can just follow a standard pattern and quickly get their classes to work with `indirect` or `polymorphic`.

I've participated in multiple reviews of this paper in LEWG. I like that this paper combines wording for `indirect` and `polymorphic`; this should save LWG review time for two classes whose wording should have much in common. The paper also has good examples (Appendix B) and an extensive table of design options (Appendix C) and their relevance to interface stability.

A common pattern for abstract base classes is to make them noncopyable. `polymorphic` addresses this by permitting users' types to make their copy constructor protected. (See section "Design for polymorphic types.") In the wild, one sometimes encounters the design alternative of deleting the copy constructor and adding a `clone()` member function that returns a new object (as pointer to base class). `polymorphic` does not support this case. I'm fine with that, because there are many ways to spell `clone` in practice, while there's a single commonly accepted way to spell the copy constructor. This makes it easier for less experienced C++ developers to get their class' design right so that it works correctly with `polymorphic`.

— Strongly Favor

I think the requirement that the wrapped type of indirect is unconditionally copy constructible was underdiscussed and is enough of a drawback that I almost want to vote against the paper based on that; however, since that restriction could potentially be relaxed in a future version of the C++ standard, I think this functionality is still useful enough as is to vote in.

— Weakly Favor

I'm very conflicted on this poll. On one side, I'd really much like to have these types in C++(26). On the other, I am not fond of the current specification (some parts are really underspecified, like indirect's move constructor), and general lack of some API (direct construction from values) which really hurt usability.

For the former, I fear LWG is going to return the paper with a bunch of design questions. For the latter, I don't think these classes should be shipping `_as-is_` in C++26. At the same time, I'm unsure if one should aim at getting the current design in, and then adding the missing APIs later; or if instead that's a poor use of precious LEWG time (to review more smaller add-on papers rather than everything in one go).

— Neutral

Editor's note: An example of such follow-up papers is: [P3152R0](#) for converting constructors.

This is lightly motivated and wrapper types continue to be fraught with troubles, so I am not thrilled about the proposal.

But what makes me against is the addition of a public `valueless_after_move` function. The rationale is self inconsistent: The valueless state is not intended to be observable to the user (...) We provide a `valueless_after_move` member function that returns true if an object is in a valueless state. By adding an observable empty `indirect` is basically an optional that is engaged on construction.

Making `valueless_after_move` exposition would change my vote to neutral.

— Strongly Against

§ 3.2. Poll 2: Send "[P2872R2] Remove `wstring_convert` From C++26" to Library Working Group for C++26.

`wstring_convert` rarely does what people actually want; there are better ways to achieve the desired aims.

— Strongly Favor

It does have usage, but being alone, the help is limited.

— Weakly Favor

I continue to be unhappy about removing things with no replacement, but since the contents of the `codecvt` header are already gone, these utilities are useless.

— Neutral

§ 3.3. Poll 3: Send "[P2875R3] Undeprecate `polymorphic_allocator::destroy` for C++26" to Library Working Group for C++26.

The paper offers a careful analysis of the original deprecation, and explanation why the deprecation was not warranted. The author has extensive experience with C++ allocators. LEWG debate did not uncover any reasons not to undeprecate.

I was particularly convinced by the argument that something with a `construct` member function should also be required to have a `destroy` member function.

— Strongly Favor

Was convinced by the authors' arguments about consistency.

— Weakly Favor

There doesn't seem to be anything egregiously wrong with `polymorphic_allocator::destroy`, and the replacement is definitely worse.

— Weakly Favor

§ 3.4. Poll 4: Send "[P3029R0] Better mdspan's CTAD" to Library Working Group for C++26.

I like the change proposed by P3029R0. It's a breaking change from C++23, in that it would change the result's `extents_type`. However, it's a `_good_` breaking change.

If you're going through the trouble of passing in `integral_constant` values, you would probably be delighted to find out that `mdspan` respects them. It's also hard to imagine the kind of generic code that might pass in an arbitrary mixture of `_integral-constant-like_` and `integral` values, yet would still insist on the result having entirely run-time extents.

Note that this change doesn't add functionality. Users today who want a mix of compile-time and run-time extents, and who want to use CTAD with `mdspan`, can pass in the data handle and extents object. Such users could write a helper that turns a parameter pack of mixed `_integral-constant-like_` and `integral` values into an `extents` object with the right type.

— Strongly Favor

This is an important improvement. I do not worry about the change in semantics, since `mdspan` targets a community which frequently builds from source and is more resilient to small breakages.

— Strongly Favor

I'm concerned about the ever-increasing complexity of `mdspan`, and the possible interaction with P2389, but I think this seems like an improvement.

— Weakly Favor

Do not consider the practice to support is worthy of teaching the ugly code.

— Strongly Against

§ 3.5. Poll 5: Approve "[P3109R0] A plan for `std::execution` for C++26".

The plan makes sense. While it looks like there's a lot of design work to do for C++26, the authors have been thinking about it for a while, and I'm confident they have the ability and willpower.

The process of wording review in LWG helps understand the proposal a lot better. People who complain about not understanding P2300 would benefit from spending time in LWG with the proposal.

— Strongly Favor

I support the effort behind `std::execution`, but my understanding of the proposal is limited. I do trust the authors though, that the proposed direction is the best way forward and would like to see them make progress.

— Weakly Favor

The suggestions seem reasonable, but I don't understand enough of S/R to be sure. I hope these changes would result in a mature design that is actually suitable for standardization.

— Weakly Favor

Great plan. Quite ambitious, i hope the authors don't burn themselves out.

— Neutral

§ References

§ Informative References

[P2872R2]

Alisdair Meredith. *[Remove `wstring\_convert` From C++26](https://wg21.link/p2872r2)*. 14 September 2023. URL: <https://wg21.link/p2872r2>

[P2875R3]

Alisdair Meredith. *[Undeprecate `polymorphic\_allocator::destroy` For C++26](https://wg21.link/p2875r3)*. URL: <https://wg21.link/p2875r3>

[P3019R6]

Jonathan Coe, Antony Peacock, Sean Parent. *Vocabulary Types for Composite Class Design*.

URL: <https://wg21.link/p3019r6>

[P3029R0]

Hewill Kang. *Better mdspan's CTAD*. 24 October 2023. URL: <https://wg21.link/p3029r0>

[P3109R0]

Lewis Baker, Eric Niebler, Kirk Shoop, Lucian Radu. *A plan for std::execution for C++26*. URL:

<https://wg21.link/p3109r0>

[P3123R0]

Inbal Levi, Fabio Fracassi, Ben Craig, Nevin Liber, Billy Baker, Corentin Jabot. *2024-02 Library*

Evolution Polls. URL: <https://wg21.link/p3123r0>